

Dead Letter Queue 설계와 운영

처리 실패 격리 전략 · DLQHandler · requeueMessage 운영 절차
강의 55min + 실습 30min

DLQ

격리 → 분석 → 재큐잉

DLQHandler.ts

무한 재시도는 왜 위험한가

THE PROBLEM WITH INFINITE RETRY

재시도가 유효한 경우 (일시적 장애)

- DB 연결 오류 (일시적) → 재시도 → 성공 ✓
- 외부 API 타임아웃 → 재시도 → 성공 ✓

재시도해도 소용없는 경우 (영구 장애)

① 코드 버그

NFTIssuedProcessor — BigInt(undefined) TypeError 발생
→ 재시도해도 같은 코드 실행 → 영원히 같은 에러

② 스키마 불일치

발행자가 필드명을 tokenId → token_id 로 변경
→ event.fields['tokenId'] = undefined → parseInt → NaN
→ 재시도해도 payload 그대로 → 동일 파싱 오류

③ 비즈니스 규칙 위반

KYC 미완료 사용자 NFT 발행 요청 → AML 스크리닝 거부
→ 재시도해도 같은 사용자 → 영원히 거부

재시도만 있을 때 — 무한 루프



▲ 영구 실패 메시지 — 이 루프에서 탈출할 방법이 없다

① PEL 누적

Redis 메모리 압박

② 정상 메시지 지연

실패 메시지가 큐 막음

③ 장애 인지 불가

운영자 알림 없음

④ 버그 은폐

원인 분석 불가능

DLQ — 불량품 격리 창고 비유

DLQ IS NOT A GRAVEYARD — IT IS A TEMPORARY ISOLATION WARD

공장 생산 라인 비유

✅ 일반 라인

상품 검사 → 합격 → 출하 ✅

❌ 불량 발견

상품 검사 → 불합격 → 격리창고 🏭

DLQ = 불량품 격리 창고

실패 N회 → 원 스트림 격리 → DLQ 이동

원 스트림은 정상 메시지만 처리 (막힘 없음)

운영자 원인 분석 → 수정 → `requeueMessage()`

DLQ 전체 생명주기 — 4단계



① 정상 처리

Streams → Consumer → 성공 → XACK ✅



② DLQ 진입 (3회 실패)

`dlq.move()` → XADD dlq 스트림 · 운영팀 알림 → 원 스트림 XACK



③ 운영자 분석

`listPending()` → 원인 파악 → 코드/데이터 수정



④ DLQ 탈출 (구조)

`requeueMessage()` → 원 스트림 재투입 → XACK ✅

DLQ vs 무한 재시도 — 전면 비교

DLQ VS INFINITE RETRY — COMPREHENSIVE COMPARISON

항목	무한 재시도	DLQ (3회 후 격리)
영구 실패	무한 루프	3회 후 격리 → 원 스트림 정상화
정상 처리	실패 메시지가 계속 큐 막음	격리 후 정상 메시지 처리 재개
PEL 상태	계속 증가 (메모리 압박)	3회 후 XACK → PEL 정리
운영 인지	모니터링 없으면 영원히 모름	알림으로 즉시 인지
원인 분석	어렵거나 불가능	DLQ에 원본+실패 이유 보존
복구 방법	코드 수정 후 재시작	원인 수정 후 <code>requeueMessage</code>

대상 파일 — `internal/packages/event-engine/src/stream/DLQHandler.ts`

DLQItem 인터페이스 · move() 코드

DLQ MOVE — XADD TO DLQ STREAM + ASYNC ALERT

DLQItem 인터페이스

```
export interface DLQItem {  
  messageId: string; // 원본 Redis messageId  
  streamKey: string; // "kyobo:events"  
  groupName: string; // Consumer Group 이름  
  event: Record<string, string>; // 원본 필드 전체  
  reason: string; // 실패 원인 문자열  
  failedAt: Date;  
}
```

DLQ 스트림 키 — `kyobo:events:dlq = sourceStreamKey + ":dlq"`

원 스트림과 완전히 별개의 Redis 키. 스트림마다 DLQ가 독립 생성됨.

왜 원본 필드 전체를 보존하나? — 운영자가 나중에 "왜 실패했나" 분석할 수 있도록 컨텍스트 전부 보존. `messageId`, 이벤트 원본, 실패 시각 필수.

move() — DLQ 이동 + 운영 알림 (DLQHandler.ts:61)

```
const dlqMessageId = await this.redis.xadd(  
  this.dlqStreamKey,  
  {  
    ...item.event,  
    _originalMessageId: item.messageId,  
    _originalStream: item.streamKey,  
    _groupName: item.groupName,  
    _reason: item.reason,  
    _failedAt: item.failedAt.toISOString(),  
  }  
);  
// 알림: 비동기 — 알림 실패가 DLQ 이동을 차단하면 안 됨  
this.notifier.sendAlert(`[DLQ] ${item.messageId} 실패...`)  
  .catch(err => console.error('alert failed:', err));
```

왜 .catch()로 처리? — `await notifier.sendAlert()`면 알림 실패 시 `move()` 전체 실패 → DLQ 이동 안 됨. 알림 유실이 더 나은 트레이드오프.

listPending() — DLQ 항목 조회와 운영자 워크플로우

LISTING PENDING DLQ ITEMS — OPERATOR WORKFLOW

listPending() (DLQHandler.ts:90)

```
async listPending(count = 100): Promise<DLQItem[]> {
  const entries = await this.redis.xrange(
    this.dlqStreamKey, '-', '+', count
  );
  // '-' = 처음부터, '+' = 끝까지 (전체 범위)
  return entries.map(entry => ({
    messageId: entry.fields['_originalMessageId'] ?? entry.id,
    reason:    entry.fields['_reason'] ?? '',
    failedAt:  new Date(entry.fields['_failedAt'] ?? Date.now()),
    event:     entry.fields,
    ...
  }));
}
```

운영자 워크플로우

```
// 1. DLQ 조회
const pending = await dlqHandler.listPending(50);
console.table(pending.map(p => ({
  id:      p.messageId,
  event:   p.event['eventType'],
  reason:  p.reason,
  failed:  p.failedAt,
})));

// 2. 원인 파악 후 queueMessage 호출
const { newMessageId } =
  await dlqHandler.queueMessage(pending[0].messageId);
```

운영 절차 — listPending으로 조회 → reason 확인 → 코드/데이터 수정 → queueMessage 호출. 분석 없이 바로 queue는 금지.

requeueMessage() — 6단계 흐름

QUEUE — THE RESCUE OPERATION (DLQHandler.ts:116)

```
async requeueMessage(
  dlqMessageId: string,
  targetStreamKey?: string,
): Promise<{ newMessageId: string }> {

  // ❶ DLQ에서 메시지 조회
  const [entry] = await this.redis.xrange(
    dlqStreamKey, dlqMessageId, dlqMessageId, 1
  );
  if (!entry) throw new DLQMessageNotFoundError(dlqMessageId); // ❷

  // ❸ 목적지: 인자 ?? 원본 메타 ?? 기본값
  const targetKey = targetStreamKey
    ?? entry.fields['_originalStream'] ?? 'kyobo:events';

  // ❹ _ 메타 제거 → 비즈니스 필드만
  const requeueFields: Record<string, string> = {};
  for (const [k, v] of Object.entries(entry.fields)) {
    if (!k.startsWith('_')) requeueFields[k] = v;
  }

  // ❺ 추적 메타 추가
  requeueFields['_requeuedFrom'] = dlqMessageId;
  requeueFields['_requeuedAt'] = new Date().toISOString();

  const newMessageId = await this.redis.xadd(targetKey, requeueFields); // ❻
  await this.redis.xdel(this.dlqStreamKey, dlqMessageId); // ❼
  return { newMessageId };
}
```

6단계 흐름 요약

- ❶ DLQ에서 메시지 조회 `xrange start=end=id`
- ❷ 존재 확인 → 없으면 `DLQMessageNotFoundError`
- ❸ 목적지 결정: 인자 ?? 원본 메타 ?? 기본값
- ❹ _ 메타 필드 제거 → 비즈니스 필드만 추출
- ❺ 추적 메타 추가 `_requeuedFrom` · `_requeuedAt`
- ❻ 원 스트림에 `XADD` → 새 `messageId` 발급
- ❼ DLQ에서 `XDEL` — 영구 제거

❻→❼ 사이 크래시 — `xadd` 성공 + `xdel` 실패 → 원 스트림·DLQ 양쪽 잔존. 멍등성 or Lua로 보완.

메타 필드 _ prefix 컨벤션의 힘

UNDERSCORE PREFIX CONVENTION — AUTO FILTERING METADATA

필드 구분 원칙

분류	필드 예시	처리
비즈니스	eventType, payload, txHash, requestId	재투입 시 그대로 전달
DLQ 메타	_originalStream, _reason, _failedAt	재투입 시 제거
재투입 메타	_requeuedFrom, _requeuedAt	추적용으로 신규 추가

핵심 코드 한 줄 — `if (!k.startsWith('_')) requeueFields[k] = v;`
_ prefix 컨벤션 덕에 자동 필터링. 새 메타 필드 추가 시 코드 변경 없음.

왜 이 컨벤션이 강력한가

- 새 메타 필드 추가 시 **자동 처리** (코드 변경 X)
- Consumer가 메타와 비즈니스 데이터 **한눈에 구분**
- 디버깅 시 "이건 시스템이 추가한 거" 즉시 인지

xrange 단건 조회 패턴

```
// start = end = dlqMessageId → 정확히 1건만  
xrange(dlqStreamKey, dlqMessageId, dlqMessageId, 1)  
//  
// count=1 → 안전장치  
// Redis Streams는 messageId 중복 없지만 방어적 코드
```

방어적 코딩 — count 없으면 모든 매칭 반환. 정상 Redis라도 "혹시 모를 중복" 대비해 count=1 명시.

xadd + xdel은 원자적이지 않다

NON-ATOMIC XADD + XDEL — PARTIAL FAILURE MITIGATION

부분 실패 시나리오 — T축 타임라인

● T+0 — ⑥ xadd 성공
원 스트림에 새 메시지 적재 완료

● T+1 — 크래시
네트워크 끊김 / 프로세스 죽음

● T+2 — ⑦ xdel 실패
DLQ에 원본 그대로 잔류

결과 — 원 스트림 + DLQ 양쪽에 같은 메시지 존재
운영자가 같은 ID로 또 `requeueMessage` → 2중 처리 위험

실무 해법 3가지

해법 1: Lua 스크립트 (원자적)

```
-- Redis에서 xadd + xdel을 원자적으로 실행
local newId = redis.call('XADD', targetKey, ...)
redis.call('XDEL', dlqKey, dlqId)
return newId
```

해법 2: Consumer 측 멱등성

`_requeuedFrom` 체크 → 이미 처리한 `dlqMessageId`면 무시. 항상 멱등하게 만드는 것이 황금률.

해법 3: 트랜잭션 + 보상 작업

xadd 성공 → DB에 "requeue 진행 중" 기록 → xdel 실패 시 별도 cleanup 워커가 처리.

현실 — 운영 도구는 보통 해법 2(멱등성)에 의존. 운영자가 같은 ID로 재호출해도 첫 번째 한 번만 효과.

왜 `requeueMessage`는 수동 호출인가

WHY MANUAL REQUEUE — AUTOMATIC REQUEUE IS DANGEROUS

자동 재투입 안 하는 3가지 이유

① 영구 실패 메시지의 무한 루프

DLQ 도착 → 자동 재투입 → 또 실패
→ 또 DLQ → 자동 재투입 → ...
코드 버그가 원인이면 영원히 반복
= 시스템 리소스 낭비

② 원인 파악 없는 재투입은 위험

데이터 문제? 외부 의존성? 코드 버그? 사람이 분석 후 결정해야 함.

③ 부분 처리된 데이터 정리 필요

VASP NFT 발행 요청은 성공했는데 Core Banking 알림에서 실패 → 재투입하면 NFT 중복 발행. 온체인·거래 이력 확인 후 재투입 여부 결정해야 함.

올바른 DLQ 운영 패턴 — 4단계



① DLQ 도착 → 알림

Slack/Email → `listPending()` 실패 메시지 확인



② 원인 파악

버그 → 코드 수정 · 데이터 이상 → 정정 · 외부서비스 → 복구 확인



③ `requeueMessage(id)`

원 스트림 재투입 → DLQ 삭제



④ **Consumer 재처리** → **XACK** ✓

실패 시 → 또 DLQ → **2회 DLQ = 로직 버그 강력 의심**

이 세션에서 새로 등장한 TypeScript 문법

NEW TYPESCRIPT PATTERNS IN S11

선택적 매개변수 ?:

```
targetStreamKey?: string
```

생략 가능 → 함수 안에서 `undefined`. `string | undefined`와 동일. 점진적 기능 추가에 유리.

인라인 반환 타입

```
Promise<newMessageId: string>
```

별도 `interface` 없이 즉석 타입 명시. 호출자가 `result.newMessageId`로 접근.

배열 구조 분해

```
const [entry] = await xrange(...)
```

`xrange`는 배열 반환 → 첫 요소만 `entry`로. 비어 있으면 `entry = undefined`.

커스텀 에러 클래스

```
throw new DLQMessageNotFoundError(id)
```

호출자가 `instanceof`로 잡을 수 있음. 일반 `Error`보다 의미 명확. 에러 타입별 분기 처리 가능.

?? 연쇄 (fallback chain)

```
a ?? b ?? c
```

왼쪽부터 차례로: 인자 → 원본 메타 → 기본값. 셋 다 nullish면 마지막 사용. 우선순위 명시적 표현.

Record<string, string>

키와 값이 모두 `string`인 객체 타입. {} 빈 객체로 시작해서 동적으로 채움. Redis Streams의 `fields` 타입.

Object.entries(obj)

```
{a:1, b:2} → [['a',1],['b',2]]
```

객체를 `[key,value]` 쌍 배열로. `for...of` 루프 가능. 모던 객체 순회 패턴.

for...of + 구조 분해

```
for (const [k, v] of Object.entries(...))
```

각 `entry`가 `[key, value]` → 즉석에서 `k, v`로 분해. 일반 `for` 루프보다 깔끔.

startsWith('_')

문자열 prefix 검사. `_`로 시작 = DLQ 메타 필드 표시. 한 줄 코드로 자동 필터링.

new Date().toISOString()

ISO 8601 형식 반환. 예: 2026-05-

13T09:00:00.000Z. 로그/DB 저장 표준 형식.

강의 강조 포인트 — 핵심 암기

KEY TAKEAWAYS — MUST REMEMBER

설계 원칙

- DLQ는 무덤이 아니라 임시 격리 — `requeue`로 살려낼 수 있어야 완전한 시스템
- 운영자 수동 호출이 핵심 — 자동화 = 무한 루프 위험
- 알림 비동기 처리 — `.catch()` 패턴. 알림 실패가 DLQ 이동을 차단하면 안 됨
- DLQ 이동 후 반드시 XACK — 안 하면 원 스트림 PEL에 계속 잔류

구현 세부

- `_prefix` 컨벤션 — 메타와 비즈니스 데이터 자동 분리. 새 메타 추가 시 코드 변경 없음
- `fallback` 체인 ?? — 인자 → 원본 메타 → 기본값. 우선순위 명시
- `xrange start=end+count=1` — 단건 조회 방어적 패턴
- `xadd+xdel` 비원자성 — 부분 실패 대비 멍등성 또는 Lua 필요
- `_requeuedFrom` 체인 — 메시지 추적성 보존, 무한 루프 감지
- 커스텀 에러 클래스 — `DLQMessageNotFoundError` → 호출자 친화적 분기

한 줄 정리 — `requeueMessage`는 DLQ 격리 메시지를 운영자가 분석 후 원 스트림으로 되돌리는 6단계 함수. 수동 호출이 원칙. 핵심은 `_` 컨벤션과 멍등성 — 부분 실패에도 정합성 유지.

실습 시나리오 — 3회 실패 → DLQ → requeueMessage

EXERCISE SCENARIO — FAILURE ISOLATION AND RECOVERY

BrokenProcessor + 전체 흐름

XADD kyobo:events — NFT_ISSUED 메시지 적재

시도 1 BrokenProcessor → throw · retryCount 0 → 1

시도 2 XAUTOCLAIM → 재수신 → throw · retryCount 1 → 2

시도 3 재수신 → throw · retryCount 2 → 3

한도 초과 `retryCount(3) ≥ MAX_RETRIES(3)` → DLQ 이동 분기

`dlq.move()` XADD kyobo:events:dlq + _reason=max retries exceeded

XACK kyobo:events → PEL 정리 완료

Part 2: 운영 절차 실습

[DLQ 확인]

`listPending()` → [{ reason: 'max retries exceeded', ... }]

[재큐잉]

`requeueMessage(dmqMessageId)`

→ XADD kyobo:events * ...원본필드... _requeuedFrom=dmqId

→ XDEL kyobo:events:dlq dmqId

[재처리 확인]

`listPending()` → [] (항목 0개)

[완료 기준]

- ✓ Part 1: DLQ 항목 1건 확인
- ✓ Part 1: reason = 'max retries exceeded'
- ✓ Part 2: 재큐잉 완료 + DLQ 항목 0건
- ✓ 알림 실패해도 DLQ 이동 성공 (.catch 이해)

실습 — S11_dlq.ts · 운영 절차 4단계

EXERCISE — DLQ FULL LIFECYCLE

실습 파일 구성

- **S11_dlq.ts** — Part 1: 3회 실패 → DLQ 이동 시나리오
- **S11_dlq.ts** — Part 2: listPending + requeueMessage 운영 절차
- **BrokenProcessor** — 항상 실패하는 테스트용 프로세서 (완성 제공)

실행 — event-engine 폴더에서

```
npx ts-node src/exercises/S11_dlq.ts
```

다른 스트림으로 라우팅 (고급)

```
// 분석 후 다른 큐로 보내고 싶을 때
await dlqHandler.requeueMessage(
  dlqId,
  'kyobo:events:retry-slow' // 선택적
);
```

운영 절차 4단계 (암기)

1. `dlqHandler.listPending()`
→ DLQ 항목 확인 (`reason`, `failedAt`, `event`)
↓
2. 원인 파악
버그: 코드 수정 + 배포
데이터 이상: 정정 후 재큐잉
↓
3. `dlqHandler.requeueMessage(dlqMessageId)`
→ 원본 스트림에 재발행
→ DLQ에서 제거
↓
4. Consumer가 재처리
→ 성공 → XACK
→ 실패 → DLQ 재진입 → 즉시 알림
(2회 DLQ = 비즈니스 로직 버그 강력 의심)

주의 — `xadd` + `xdel`이 원자적이지 않은 이유와 멍등성으로 보완하는 방법을 설명할 수 있어야 함.